Approvo. l'unico punto è il seguente, sul quale ti chiedo una valutazione critica :

Nome fisso e concorrenza
In presenza di un container omonimo errore esplicito anzichè rimozione forzata ?
valutare in base a :
i containers hanno nomi codificati e un container omonimo dovrebbe riferirsi ad una precedente esecuzione del progetto: esiste la possibilità di riutilizzarlo ?
Si, non è stabilito che i containers siano sempre effimeri.

Hai ragione: se il nome codificato identifica una precedente esecuzione dello stesso progetto e il container può essere persistente, l'errore per il solo fatto che esista è troppo restrittivo. La rimozione forzata automatica è però altrettanto inappropriata.

La politica migliore è **riutilizzo controllato**: verificare identità, stato e compatibilità del container, quindi decidere se avviarlo, collegarsi alla sessione esistente oppure richiederne la ricreazione.

1. Un container omonimo può essere riutilizzato

Docker consente di riavviare un container fermo, anche collegando terminale e input:

```
docker start --attach --interactive "$CONTAINER_NAME"
```

Il riavvio usa il container già creato, non ne crea uno nuovo.[1] [2]

Questo è utile per conservare il filesystem interno del container. Non significa invece riprendere la memoria dei processi o la sessione shell precedente: il processo principale viene eseguito nuovamente.

La distinzione operativa è:

Stato	Possibile riutilizzo
Container assente	Creazione
Container fermo	Riavvio del suo processo principale
Container in esecuzione	Collegamento al processo principale oppure esecuzione di un nuovo comando
Container incompatibile con la configurazione richiesta	Ricreazione, oppure uso consapevole della vecchia configurazione

Per un container già in esecuzione, `attach` collega il terminale al processo principale; `exec` avvia invece un processo aggiuntivo. Sono operazioni diverse, non due modi equivalenti di "riprendere" la sessione. [3] [4]

2. Il nome codificato è un indizio, non una verifica sufficiente

La tua osservazione è corretta: nella normale gestione di `docker-dev`, un container omonimo dovrebbe appartenere al progetto. Ma basarsi esclusivamente sul nome lascia aperti casi come:

- due progetti che producono lo stesso nome normalizzato;
- un progetto spostato o rinominato;
- un container creato manualmente;
- una modifica delle convenzioni di naming.

Proporrei quindi di aggiungere label applicative, per esempio:

```
labels:
  io.docker-dev.managed: "true"
  io.docker-dev.project-id: "<identità-progetto>"
  io.docker-dev.role: dev
  io.docker-dev.runtime-hash: "<impronta-configurazione>"
```

Queste sono label proposte per il nostro tool. Il nome serve a trovare il container; le label servono a stabilire se possiamo gestirlo.

Se l'identità non corrisponde, allora l'errore è corretto: non bisogna riutilizzare né eliminare automaticamente il container.

3. Essere dello stesso progetto non implica essere aggiornato

Un container fermo potrebbe appartenere al progetto corretto ma essere stato creato con:

- un'altra immagine;
- mount differenti;
- altre variabili d'ambiente;
- porte o risorse diverse;
- un altro comando iniziale.

Riavviarlo non applica la nuova configurazione dichiarata nei file runtime.

Il controllo deve quindi distinguere:

```
appartenenza al progetto
          ≠
compatibilità con l'esecuzione richiesta
```

L'impronta runtime dovrebbe rappresentare la configurazione di creazione risolta, includendo almeno immagine effettiva, mount, ambiente, porte, utente, working directory, entrypoint e

comando iniziale.

Non deve essere un hash dell'intero contenuto del progetto: cambiare un file sorgente nel bind mount non dovrebbe obbligare a ricreare il container.

4. Persistenza e modalità di esecuzione sono due scelte separate

Dobbiamo distinguere:

Dimensione	Alternative
Lifecycle	Effimero oppure persistente
Uso	Sessione in foreground oppure container residente

Per esempio, è perfettamente sensato avere:

```
container persistente
+
shell in foreground
+
container fermo quando si esce dalla shell
```

Il container resta disponibile per il prossimo avvio, pur non rimanendo sempre in esecuzione.

La CR dovrebbe quindi dire "effimero per default, se questa è la scelta approvata", non "sempre effimero".

5. Due modi per realizzare il riutilizzo

Opzione A — Conservare `compose run`

Si può creare un container one-off senza `--rm`:

```
docker compose "${compose_args[@]}" \
  run \
  --name "$CONTAINER_NAME" \
  --service-ports \
  dev
```

Il container rimane dopo l'uscita. Ma una successiva esecuzione di `compose run` crea un nuovo container: non è un comando di riavvio di quello omonimo. [5]

L'orchestratore deve quindi gestire direttamente:

```
creazione → compose run
riavvio   → docker start
attach    → docker attach
exec      → docker exec
```

È fattibile, ma introduce un lifecycle personalizzato sopra i container one-off.

Opzione B — Usare un normale container del servizio Compose

Se il riutilizzo è una funzionalità stabile del tool, questa è la soluzione che preferisco per la modalità persistente.

Il container `dev` viene creato come container del servizio:

```
docker compose "${compose_args[@]}" create dev
```

Compose dispone di una gestione esplicita della creazione e delle politiche di ricreazione dei container di servizio.[6]

Successivamente si avvia la sessione sul container risolto, per esempio con:

```
docker start --attach --interactive "$CONTAINER_NAME"
```

Per i normali container dei servizi, `docker compose exec dev ...` è inoltre l'interfaccia appropriata per eseguire comandi aggiuntivi quando il container è attivo.[4]

Questa strada richiede di rivedere la precedente indicazione "creare `dev` esclusivamente con `run`": era adeguata al caso effimero, non a tutti i lifecycle possibili.

6. Politica proposta per il container omonimo

Situazione	Comportamento proposto
Non esiste	Creare secondo il lifecycle richiesto
Esiste, ma non appartiene al progetto	Errore; nessuna modifica
È fermo, persistente e compatibile	Riavviarlo
È attivo e compatibile	Segnalare la sessione attiva; consentire `attach` o `exec` espliciti
È fermo ma incompatibile	Mostrare le differenze e richiedere ricreazione esplicita
È attivo ma incompatibile	Non interromperlo automaticamente; richiedere stop e ricreazione espliciti
È persistente, ma viene richiesta una sessione effimera con lo stesso nome	Conflitto di lifecycle da risolvere esplicitamente

Non farei `attach` automatico a una sessione già attiva: un secondo terminale collegato al medesimo processo principale condivide la sessione, non ottiene una shell indipendente.[3]

7. Il caso dei comandi diversi

Questo è uno dei punti più importanti.

Se il container era stato creato per aprire una shell, questa richiesta:

```
docker-dev --target=Python/foo -- pytest -q
```

non può essere implementata semplicemente riavviando il container: `start` esegue il comando iniziale, non lo sostituisce.

Occorre scegliere fra:

- `exec`, se il container è già attivo;
- un container one-off distinto per il task;
- ricreazione del container con il nuovo comando.

Proporrei di separare il container persistente di sviluppo dai task one-off, evitando di trasformare ogni cambio di comando in una ricreazione della sessione principale.

Raccomandazione

Ritengo quindi corretta la tua obiezione e modificherei la politica precedente:

- niente errore per la sola esistenza del nome;
- niente rimozione forzata automatica;
- riavvio automatico consentito per un container persistente, fermo, appartenente al progetto e compatibile;
- azioni esplicite per sessioni già attive, cambi di configurazione o cambi di lifecycle;
- `compose run --rm` per task/sessioni effimere;
- normale container di servizio Compose per la modalità persistente.

Questo non reintroduce il backend legacy: le definizioni restano Compose. Significa soltanto che `docker-dev` usa l'operazione adatta al lifecycle richiesto, invece di assumere che ogni invocazione debba creare e distruggere un container.

⁂

1. https://docs.docker.com/reference/cli/docker/container/start/
2. https://docs.docker.com/reference/cli/docker/container/create/
3. https://docs.docker.com/reference/cli/docker/container/attach/
4. https://docs.docker.com/reference/cli/docker/compose/exec/
5. https://docs.docker.com/reference/cli/docker/compose/run/
6. https://docs.docker.com/reference/cli/docker/compose/create/
7. https://docs.docker.com/reference/cli/docker/compose/up/

8. https://docs.docker.com/reference/cli/docker/compose/
9. https://docs.docker.com/reference/cli/docker/container/run/
10. https://docs.docker.com/compose/support-and-feedback/faq/
11. https://docs.docker.com/engine/containers/run/
12. https://docs.docker.com/reference/compose-file/services/
13. https://docs.docker.com/reference/cli/docker/compose/rm/
14. https://docs.docker.com/compose/releases/release-notes/
15. https://docs.docker.com/reference/cli/docker/container/stop/